

Search and A* Laboratory

Warehouse Robot Navigation

This report covers the submission items specified in the Search hands-on: search formulation, agent design, final Python program, LLM prompt, tests, BFS/A* comparison, heuristic investigation, and reflection.

1. Search problem formulation

State S: the robot's current grid position, represented by (row, column).

Actions A: Up, Down, Left and Right.

Transition T: a valid action changes the position by one grid cell.

Initial state s0: the cell marked S.

Goal G: the cell marked G.

Cost c: 1 for every move.

A state is fully specified by the robot's position because the warehouse is fixed. A move is invalid if it leaves the grid or enters #. The search problem is deterministic because a given state and action always produce the same result. A solution is a sequence of valid moves that takes the robot from S to G.

2. Agent design

I used grid coordinates as states and the supplied ASCII map as the environment. The successor function checks the four possible moves and keeps only free cells. For A*, the frontier is a priority queue containing states ordered by $f(n)=g(n)+h(n)$. A parent dictionary stores how each state was reached so the final path can be reconstructed.

The program reports whether a solution exists, the sequence of actions, the path length, and the number of states expanded.

3. LLM prompt and inspection

The exact prompt is stored separately in `01_astar_prompt.txt`. I specified the warehouse, movement rules, unit movement cost, Manhattan heuristic, A* requirements, path reconstruction, and the required output. After generating the code, I checked the state representation, neighbour generation, g/h/f calculations, priority queue, duplicate-state handling, and path reconstruction before running the tests.

4. Original warehouse result

A* found a path of **40** moves and expanded **64** states. One valid sequence of actions was:

Right → Right → Right → Right → Down → Down → Down → Down → Right → Right → Right → Right → Right → Right → Right → Right → Up → Up → Left → Left → Left → Left → Left → Left → Up → Up → Right → Right → Right → Right → Right → Right → Right → Right → Down → Down → Down → Down → Down → Down

5. Test results

Test	A* found	A* length	A* expanded
Trivial one-step	True	1	2
No solution	False	—	9
Alternative paths	True	4	5

The trivial case returned the required one-step path. The no-solution map returned failure after the reachable states were exhausted. The alternative-path test returned a valid shortest path of length 4.

6. A* implementation inspection

State: tuple (row, column).

Action: one of the four movement directions.

Transition: neighbour cell that is inside the grid and not #.

Goal test: current == goal.

g(n): stored in the g_cost dictionary.

$h(n)$: Manhattan distance function.

$f(n)$: computed as $g+h$ when a state is pushed into the heap.

Frontier: Python heapq priority queue.

Repeated exploration: better-g checks plus stale-entry skipping.

Path reconstruction: parent dictionary followed backwards from G.

7. BFS vs A*

Measure	BFS	A*
Solution found	Yes	Yes
Path length	40	40
States expanded	64	64

For this particular warehouse and tie-breaking order, BFS and A* both expanded 64 states and produced a 40-move path. A* still uses additional information through $h(n)$, but on this small map that information did not reduce the measured number of expansions with the chosen implementation and ordering.

8. Heuristic investigation

Heuristic	Solution	Path length	States expanded
$h(n)=0$	True	40	64
Manhattan	True	40	64
Euclidean	True	40	64
$2 \times \text{Manhattan}$	True	40	67

Manhattan distance is appropriate because the robot can move only horizontally or vertically. Ignoring obstacles, the Manhattan distance is the minimum number of grid moves needed to line up with the goal, so it does not overestimate the actual remaining cost. $h(n)=0$ behaves like uniform-cost search for unit costs. Euclidean distance also gave the same path length in this experiment. Multiplying Manhattan distance by 2 made the heuristic more aggressive and resulted in 67 expansions on this implementation, while the returned path still happened to have length 40. The experiment shows why the heuristic affects the search order and why the usual optimality guarantee depends on heuristic properties such as admissibility.

9. Task 7 - Evaluation of the LLM-generated agent

1. What parts of the generated code were correct immediately?

The basic grid representation, four-direction neighbour generation, Manhattan heuristic, priority queue and parent-based path reconstruction matched the specification.

2. Did you find any bugs or design problems?

I checked the handling of repeated states and old heap entries. The final version keeps the best known g-cost for each state and ignores stale queue entries.

3. How did you discover those problems?

I compared the implementation with the A* steps from the lab and ran the known-answer test cases.

4. Did the LLM use terminology or data structures that you did not understand?

The priority queue and the relationship between g, h and f were the parts I had to inspect most carefully because they determine the expansion order.

5. Did you modify the LLM-generated code?

Yes. I made the path-cost handling and stale-entry check explicit and added the reporting needed by the lab.

6. Which tests were most useful?

The one-step case checked the basic transition logic, the no-solution case checked termination, and the alternative-path case checked shortest-path behaviour.

7. Could you have trusted the program without testing it?

No. A search program can print a plausible path while still using an incorrect expansion rule or failing on a corner case.

8. What did you understand about A* that you did not understand before implementing it?

I understood more clearly that A* balances the distance already travelled with an estimate of the remaining distance through $f(n)=g(n)+h(n)$.

10. Final reflection

Why formulate the search problem before writing the algorithm?

It makes the state, action, transition, goal and cost definitions explicit. That prevents the implementation from solving a different problem by mistake.

In what sense is A* informed?

A* uses $h(n)$, an estimate of the cost from the current state to the goal, in addition to the cost already accumulated.

Why does the choice of heuristic matter?

It changes the order in which states are explored. A useful admissible heuristic can guide the search toward the goal while preserving the usual shortest-path guarantee.

What did the LLM contribute?

It helped convert the written search specification into a Python implementation and made the initial coding stage faster.

What could go wrong if the code was accepted without testing?

The program could contain an incorrect heuristic, wrong path reconstruction, repeated exploration, or failure-handling problem and still produce output that looks reasonable.